1 Data Encryption Standard (DES)

The **Data Encryption Standard** is the symmetric-key block cipher published by NIST (then NBS) in 1977, based on IBM's Lucifer. It encrypts a **64-bit block** with a **56-bit key** through **16 Feistel rounds**. The key is normally supplied as 64 bits, of which 8 are parity bits that are dropped, and a key schedule derives sixteen **48-bit round keys** $K_1, \ldots, K_{16}$ from it. Decryption is done by the **reverse cipher**, which is the same algorithm with the round keys used in the order $K_{16}, \ldots, K_1$. DES has since been replaced by AES because its key is too short, but its structure is the standard example of a Feistel cipher (Section **??**).

1.1 DES structure ★

★ **In the papers:** 2024–25 Q6 (5 marks): Feistel structure, block diagrams of the DES algorithm and the round function, pseudocode for the S-box substitution and key generation.

Encryption consists of an **initial permutation** (IP) of the 64-bit plaintext, sixteen rounds, and a **final permutation** (FP). After IP the block is split into a left half L_0 and a right half R_0 of 32 bits each. Each round i takes (L_{i-1}, R_{i-1}) and produces (L_i, R_i) with a Feistel **mixer** followed by a **swapper**:

$$L_i = R_{i-1}, \qquad R_i = L_{i-1} \oplus f(R_{i-1}, K_i),$$

where f is the DES function described below. Round 16 has no swapper, and after it the halves are combined and passed through FP to give the 64-bit ciphertext. The block diagrams of the whole algorithm, of one round and of the key generator are drawn in the exam answer at the end of the key-generation subsection.

1.2 Initial and final permutations

IP and FP are keyless **straight P-boxes** (64×64) that are inverses of each other: $\text{FP} = \text{IP}^{-1}$. The IP table is read row by row; entry j is the input bit that goes to output position j.

Initial permutation

58	50	42	34	26	18	10	02
60	52	44	36	28	20	12	04
62	54	46	38	30	22	14	06
64	56	48	40	32	24	16	08
57	49	41	33	25	17	09	01
59	51	43	35	27	19	11	03
61	53	45	37	29	21	13	05
63	55	47	39	31	23	15	07

Final permutation

40	08	48	16	56	24	64	32
39	07	47	15	55	23	63	31
38	06	46	14	54	22	62	30
37	05	45	13	53	21	61	29
36	04	44	12	52	20	60	28
35	03	43	11	51	19	59	27
34	02	42	10	50	18	58	26
33	01	41	09	49	17	57	25

So output bit 1 of IP is input bit 58, and input bit 1 lands in output position 40; correspondingly the FP table has 40 in position 1 (FP puts it back).

Why they add no security. Both permutations are fixed, keyless and published. An attacker who knows a plaintext can apply IP to it himself, and can apply $\text{IP} = \text{FP}^{-1}$ to the ciphertext, which strips both permutations off and leaves exactly the 16 rounds to attack. IP and FP have no cryptographic significance; they are usually explained as a convenience for loading 8-bit bytes into 1970s hardware.

WORKED EXAMPLE : initial permutation (Forouzan style)

Q. Find the output of the initial permutation for the input `0x0002 0000 0000 0001` and show that FP restores it.
A. The input has only two 1-bits: hex digit 4 is 2 = 0010, i.e. bit $12 + 3 =$ **15**, and the last bit, **64**. In the IP table, 15 is entry 63 and 64 is entry 25, so the output has 1s exactly in positions 25 and 63:

$$\text{IP}(0002000000000001) = \texttt{0x0000 0080 0000 0002}.$$

(Bit 25 is the leading bit of hex digit 7, giving 8; bit 63 is the third bit of hex digit 16, giving 2.)
Check: in the FP table, entry 15 is 63 and entry 64 is 25, so FP moves bits 63 and 25 back to positions 15 and 64.

1.3 Rounds: mixer, swapper and the reverse cipher

Each round is a Feistel round (Section **??**): the **mixer** replaces L_{i-1} by $L_{i-1} \oplus f(R_{i-1}, K_i)$ and leaves R_{i-1} alone, and the **swapper** exchanges the halves. The mixer is invertible because of the XOR: given the output, the receiver still has R_{i-1}, recomputes $f(R_{i-1}, K_i)$ and XORs it away. The mixer and the swapper are each **self-inverse**: applying either twice (the mixer with the same key) gives back the input.

Why decryption is the same algorithm with keys reversed. Write encryption as the sequence of operations

$$\text{IP},\ M_{K_1},\ S,\ M_{K_2},\ S,\ \ldots,\ M_{K_{15}},\ S,\ M_{K_{16}},\ \text{FP},$$

where M_K is the mixer with key K and S the swapper. The inverse of a sequence applies the inverses of its steps

in reverse order, and here $\mathrm{FP}^{-1} = \mathrm{IP}$, $M_K^{-1} = M_K$, $S^{-1} = S$ and $\mathrm{IP}^{-1} = \mathrm{FP}$. Decryption is therefore

$$\mathrm{IP},\ M_{K_{16}},\ S,\ M_{K_{15}},\ S,\ \ldots,\ S,\ M_{K_1},\ \mathrm{FP}.$$

This is exactly the encryption algorithm with the round keys in the order $K_{16}, \ldots, K_1$. The match works only because round 16 has no swapper: with a swapper there, the inverse would start with S and would no longer have the same shape as the cipher. The last round has no swapper so that the reverse cipher is the cipher itself with reversed round keys. (Equivalently, Stallings lets all 16 rounds swap and adds a 32-bit swap before FP; the two descriptions give the same ciphertext.)

The slides' pseudocode for the cipher (Forouzan):

```
Cipher (plainBlock[64], RoundKeys[16, 48], cipherBlock[64])
{
    permute (64, 64, plainBlock, inBlock, InitialPermutationTable)
    split (64, 32, inBlock, leftBlock, rightBlock)
    for (round = 1 to 16)
    {
        mixer (leftBlock, rightBlock, RoundKeys[round])
        if (round ≠ 16) swapper (leftBlock, rightBlock)
    }
    combine (32, 64, leftBlock, rightBlock, outBlock)
    permute (64, 64, outBlock, cipherBlock, FinalPermutationTable)
}
```

Here mixer does $T \leftarrow f(\text{rightBlock}, K)$, leftBlock $\leftarrow$ leftBlock $\oplus\ T$, and swapper exchanges the two blocks through a temporary copy.

1.4 The DES function $f(R_{i-1}, K_i)$ ★

The DES function is the heart of the cipher and the only place where the key enters. It applies the 48-bit round key to the 32-bit right half and returns 32 bits, through four steps.

1. Expansion P-box (32 → 48). R_{i-1} has 32 bits but K_i has 48, and the S-boxes need 6-bit inputs, so R_{i-1} is expanded first. It is viewed as eight 4-bit sections; each section is widened to 6 bits by adding, on its left, the last bit of the previous section and, on its right, the first bit of the next section (cyclically, so section 1 borrows bit 32 and section 8 borrows bit 1):

32	01	02	03	04	05		04	05	06	07	08	09
08	09	10	11	12	13		12	13	14	15	16	17
16	17	18	19	20	21		20	21	22	23	24	25
24	25	26	27	28	29		28	29	30	31	32	01

The 16 bits that appear twice are the first and fourth bits of every section (1, 4, 5, 8, ..., 29, 32). This duplication is deliberate: a bit at the edge of a section reaches two neighbouring S-boxes, so changing it changes the input of two S-boxes, which is what makes the avalanche grow quickly. Being an expansion P-box, this step is not invertible, which a Feistel cipher can afford.

2. Whitener. The 48-bit expanded half is XORed with the 48-bit round key K_i. This is the only point at which the key influences the round.

3. S-boxes (48 → 32). The 48 bits are cut into eight 6-bit groups, and group j enters S-box S_j, which outputs 4 bits. The S-boxes do the real mixing (confusion), being the only nonlinear part of DES. Each S-box is a table of 4 rows and 16 columns. For a 6-bit input $b_1 b_2 b_3 b_4 b_5 b_6$:

$$\text{row} = (b_1 b_6)_2 \in \{0..3\}, \qquad \text{column} = (b_2 b_3 b_4 b_5)_2 \in \{0..15\},$$

and the cell is the 4-bit output. The outer bits pick one of four substitutions and the middle bits are substituted by it; every row of every S-box is a permutation of 0..15 (checked by script), so each output value has exactly 4 preimages and the box as a whole is not invertible. The eight S-boxes are different. S-box 1:

S1	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	14	4	13	1	2	15	11	8	3	10	6	12	5	9	0	7
1	0	15	7	4	14	2	13	1	10	6	12	11	9	5	3	8
2	4	1	14	8	13	6	2	11	15	12	9	7	3	10	5	0
3	15	12	8	2	4	9	1	7	5	11	3	14	10	0	6	13

4. Straight P-box (32 → 32). The 32 S-box output bits are permuted by a straight P-box; entry j is the input bit sent to output j, so the seventh input bit becomes the second output bit:

16	07	20	21	29	12	28	17
01	15	23	26	05	18	31	10
02	08	24	14	32	27	03	09
19	13	30	06	22	11	04	25

Its job is diffusion: the 4 output bits of each S-box are sent to positions that feed different S-boxes in the next round.

WORKED EXAMPLE : S-box lookup

▸ **Likely this year:** "The input to S-box 1 is 100011. What is the output?" (Forouzan's standard S-box question.)
Q. (a) The input to S-box 1 is 100011. What is the output? (b) The input to S-box 8 is 000000. What is the output?
A. (a) Outer bits $b_1 b_6$ = 1 1, so row = $(11)_2$ = 3. Middle bits $b_2..b_5$ = 0001, so column = 1. In S1, row 3, column 1 holds 12, so the output is **1100**.
(b) Outer bits 0 0 ⇒ row 0; middle bits 0000 ⇒ column 0. S8 row 0 begins 13 2 8 4 ..., so the output is 13 = **1101**.
More S1 lookups for practice: 000000 → row 0, col 0 → 14 = 1110; 111111 → row 3, col 15 → 13 = 1101; 011011 → row 1, col 13 → 5 = 0101.

Trap: row and column bits

The row is the *first and last* bit (not the first two), and the column is the *middle four*. Numbering starts at 0 for both rows and columns. Writing the answer as a decimal (12) without converting it to 4 bits (1100) loses the mark that the output is a 4-bit word.

WORKED EXAMPLE : one full DES round (Forouzan's trace: P = 123456ABCD132536, key AABB09182736CCDD)

Q. After the initial permutation, L_0 = 14A7D678 and R_0 = 18CA18AD; the first round key is K_1 = 194CD072DE8C. Compute $f(R_0, K_1)$, L_1 and R_1.
A. R_0 = 0001 1000 1100 1010 0001 1000 1010 1101. Expansion (each 4-bit section borrows its neighbours' edge bits) gives $E(R_0)$ = 8F16540F155A; XOR with K_1 gives 965A847DCBD6. The eight 6-bit groups go through the S-boxes:

S-box	1	2	3	4	5	6	7	8
$E(R_0)$ group	100011	110001	011001	010100	000011	110001	010101	011010
K_1 group	000110	010100	110011	010000	011100	101101	111010	001100
XOR (input)	100101	100101	101010	000100	011111	011100	101111	010110
row, col	3, 2	3, 2	2, 5	0, 2	1, 15	0, 14	3, 7	0, 11
output	8	10	15	14	6	5	7	14

S-box output = 8AFE657E; the straight P-box gives $f(R_0, K_1)$ = 4EDF35EC. Then

$$L_1 = R_0 = \text{18CA18AD}, \qquad R_1 = L_0 \oplus f = \text{14A7D678} \oplus \text{4EDF35EC} = \text{5A78E394}.$$

Check: this is the round-1 output in Forouzan's table; continuing all 16 rounds (round 16 output 19BA9212 CF26B472, no swap) and FP gives the ciphertext C0B7A8D05F3A829C, and running the same program with the keys reversed returns the plaintext.

1.5 Key generation ★

The round-key generator must turn one 56-bit cipher key into sixteen different 48-bit round keys, cheaply and so that every key bit influences many rounds. It works in four steps.

1. **Parity drop (64 → 56).** The key is supplied as 64 bits in which bits 8, 16, 24, ..., 64 are parity bits (each key byte has odd parity). A compression permutation (PC-1) discards these 8 bits and permutes the remaining 56.
2. **Split.** The 56 bits are split into two 28-bit halves C_0 and D_0.
3. **Shift.** In round i both halves are circularly shifted left, by **1 bit in rounds 1, 2, 9 and 16** and by 2 bits in all other rounds, giving C_i and D_i.
4. **Compression P-box (56 → 48).** $C_i \| D_i$ passes through a compression P-box (PC-2) that blocks 8 of the 56 bits and permutes the other 48; the result is K_i.

Why it is built this way. Because the halves are rotated before each compression, a different selection and arrangement of key bits reaches the S-boxes in every round; each key bit ends up in 12 to 15 of the 16 round keys (about 14 on average, counted by script). The shift amounts add up to $4 \times 1 + 12 \times 2 = 28$, a full rotation of a 28-bit half, so $C_{16} = C_0$ and $D_{16} = D_0$. This lets the decryption side generate $K_{16}, \ldots, K_1$ on the fly by *right* shifts starting from the original halves.

WORKED EXAMPLE : first round key (same key as the trace above)

Q. For the key AABB09182736CCDD, find C_0, D_0, C_1, D_1 and K_1.
A. Parity drop gives the 56-bit key C3C033A33F0CFA, so C_0 = C3C033A, D_0 = 33F0CFA. Round 1 shifts by 1 bit:

$$C_0 = 1100\,0011\,1100\,0000\,0011\,0011\,1010 \xrightarrow{\text{rotl 1}} C_1 = 1000\,0111\,1000\,0000\,0110\,0111\,0101 = 8780675$$
$$D_0 = 0011\,0011\,1111\,0000\,1100\,1111\,1010 \xrightarrow{\text{rotl 1}} D_1 = 0110\,0111\,1110\,0001\,1001\,1111\,0100 = 67E19F4$$

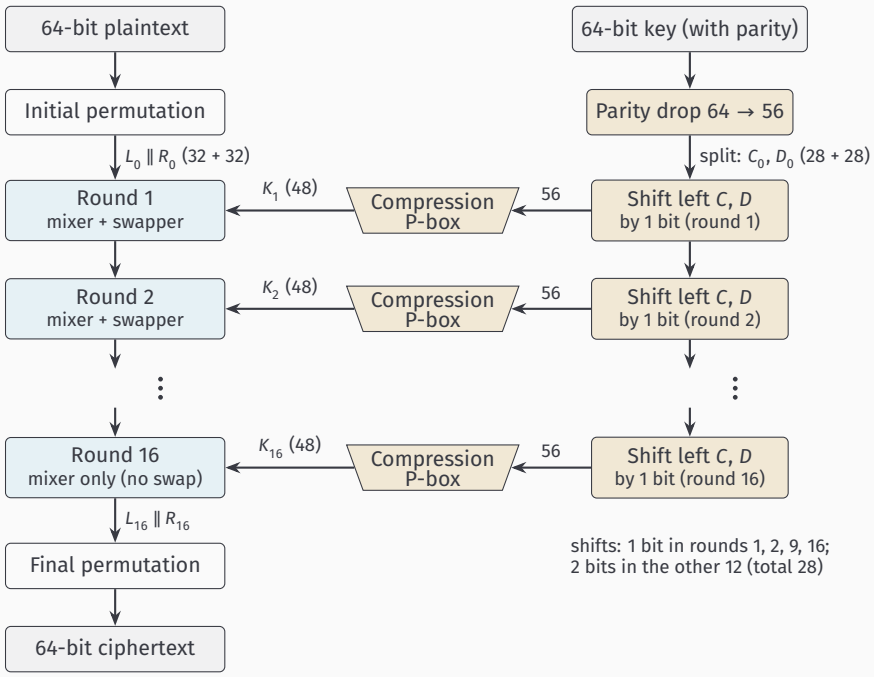
The compression P-box applied to $C_1 \| D_1$ gives K_1 = 194CD072DE8C, the key used in the round trace above. *Check:* in each half the leading bit moved to the end (e.g. C_0 starts with 1 and C_1 ends with 1).

WORKED EXAMPLE : 2024–25 Q6 (5 marks), answer as it should be written

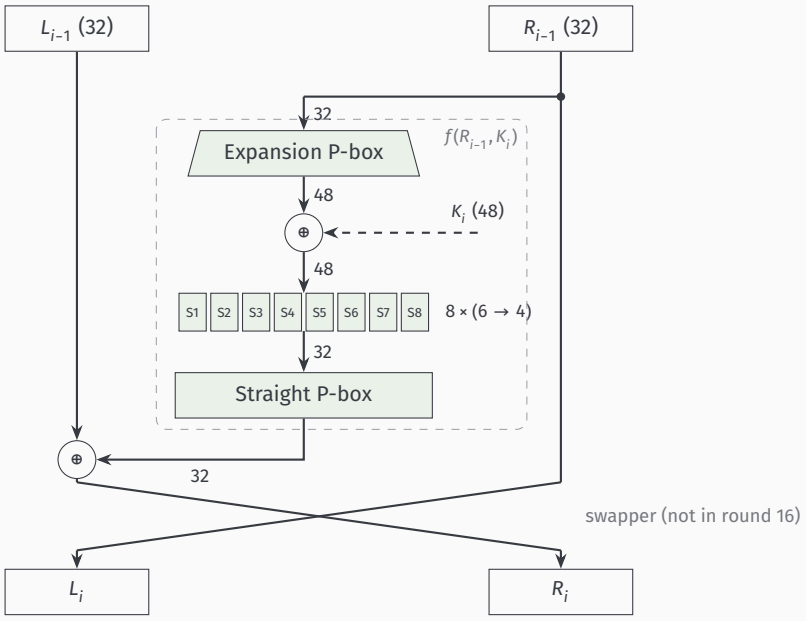
Q. In DES, what Feistel structure is prevalent? Use flow diagrams and key generation to define, with block diagrams, (1) the algorithm for DES and (2) the algorithm for the round function. How would you write the pseudocode for the substitution block and key generation?
A. Feistel structure. DES is a Feistel cipher: the 64-bit block (after IP) is split into 32-bit halves, and each of the 16 rounds applies a mixer and a swapper,

$$L_i = R_{i-1}, \qquad R_i = L_{i-1} \oplus f(R_{i-1}, K_i), \qquad i = 1..16 \text{ (no swap in round 16).}$$

All noninvertible parts (expansion P-box, 6 → 4 S-boxes) are inside f, whose output is only XORed in. Since $R_{i-1} = L_i$ is available to the receiver, $L_{i-1} = R_i \oplus f(L_i, K_i)$; so f never has to be inverted, and decryption is the same algorithm with keys $K_{16}, \ldots, K_1$.
(1) DES algorithm and key generation.



(2) Round i and the round function $f(R_{i-1}, K_i)$.



Inside f: the expansion P-box widens R_{i-1} from 32 to 48 bits, the whitener XORs in K_i, eight S-boxes reduce 48 bits to 32 (row = outer bits, column = middle 4 bits), and a straight P-box permutes the 32 bits.
Pseudocode (Forouzan style; bit arrays are indexed from 1).

```
substitute (inBlock[48], outBlock[32], SubstitutionTables[8, 4, 16])
{
  for (i = 1 to 8)
  {
    b ← (i − 1) × 6                          // start of group i
    row ← 2× inBlock[b+1] + inBlock[b+6]
    col ← 8× inBlock[b+2] + 4× inBlock[b+3]
          + 2× inBlock[b+4] + inBlock[b+5]
    value ← SubstitutionTables[i][row][col]
    o ← (i − 1) × 4
    outBlock[o+1] ← value / 8;  value ← value mod 8
    outBlock[o+2] ← value / 4;  value ← value mod 4
    outBlock[o+3] ← value / 2;  value ← value mod 2
    outBlock[o+4] ← value
  }
}
```

```
Key_Generator (keyWithParities[64], RoundKeys[16, 48], ShiftTable[16])
{
  permute (64, 56, keyWithParities, cipherKey, ParityDropTable)
  split (56, 28, cipherKey, leftKey, rightKey)
  for (round = 1 to 16)
  {
    shiftLeft (leftKey, ShiftTable[round])
    shiftLeft (rightKey, ShiftTable[round])
    combine (28, 56, leftKey, rightKey, preRoundKey)
    permute (56, 48, preRoundKey, RoundKeys[round], KeyCompressionTable)
  }
}
```

ShiftTable = [1, 1, 2, 2, 2, 2, 2, 2, 1, 2, 2, 2, 2, 2, 2, 1]; shiftLeft is a circular left shift of a 28-bit half.

Check: the substitute routine reproduces the S-box output 8AFE657E of the round trace above (tested by script).

★ MUST KNOW : DES at a glance

64-bit block, 56-bit key (64 with parity), 16 Feistel rounds, 48-bit round keys. Round: $L_i = R_{i-1}$, $R_i = L_{i-1} \oplus f(R_{i-1}, K_i)$, no swap in round 16. f: expansion 32 → 48, XOR K_i, 8 S-boxes 6 → 4 (row = $b_1 b_6$, column = $b_2..b_5$), straight P-box 32 → 32. Keys: parity drop 64 → 56, split 28 + 28, rotate left 1 bit in rounds 1, 2, 9, 16 and 2 bits otherwise, compression 56 → 48. Decryption: same algorithm, keys $K_{16}..K_1$.

1.6 DES analysis: avalanche and completeness

Two properties are expected of a good block cipher. The **avalanche effect** means that a small change in the plaintext (or the key) should change many ciphertext bits, about half of them. **Completeness** means that each ciphertext bit should depend on many plaintext bits. In DES both come from the combination of S-boxes (confusion) and P-boxes, including the expansion that feeds each edge bit to two S-boxes (diffusion), and tests show DES is strong in both.

WORKED EXAMPLE : avalanche in DES (Forouzan's example)

Q. Encrypt the plaintexts 0000000000000000 and 0000000000000001 (differing in one bit) with the key 22234512987ABB23. How many ciphertext bits differ?

A. The ciphertexts are `4789FD476E82A5F1` and `0A4ED5C15A63FEA3`; their XOR has **29** one-bits, so a 1-bit change of the plaintext changed 29 of the 64 ciphertext bits. The number of differing bits after each round (script) shows how fast the difference spreads:

round	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
bits differing	1	6	20	29	30	33	32	29	32	39	33	28	30	31	30	29

After only 3 to 4 rounds the difference is already near the ideal of 32; the remaining rounds keep it there.

1.7 Weaknesses of DES

Design of the S-boxes and P-boxes. Critics point out a few irregularities, none of which has led to a practical attack:

- In S-box 4, the last three output bits can be derived in the same way as the first output bit by complementing some of the input bits.
- Two specially chosen inputs to the S-box array can produce the same output.
- The same output of a single round can be obtained by changing bits in only three neighbouring S-boxes.
- The initial and final permutations have no security benefit, and in the expansion P-box the first and fourth bits of every 4-bit section are repeated.

Key size. The real weakness is the 56-bit key. A brute-force attack on one ciphertext block must try up to $2^{56} \approx 7.2 \times 10^{16}$ keys. At 10^6 keys per second on each of 10^6 parallel chips, i.e. 10^{12} keys per second, the whole key space takes $2^{56}/10^{12} \approx 72\,058$ seconds ≈ **20 hours** (10 hours on average, since the key is found halfway on average). Networked computers simulate parallel hardware: the slides quote 3500 Internet computers finding an RSA Laboratories challenge key in 120 days, so 42 000 machines would need $3500 \times 120/42\,000 = 10$ days.

Weak keys. A **weak key** is one for which all 16 round keys are the same. This happens when, after the parity drop, each 28-bit half is all 0s or all 1s, because rotating such a half changes nothing. There are 4 weak keys (verified by script: each gives a single distinct round key):

Key before parity drop (64 bits)	Actual key: C_0, D_0 (hex)
`0101 0101 0101 0101`	`0000000 0000000`
`1F1F 1F1F 0E0E 0E0E`	`0000000 FFFFFFF`
`E0E0 E0E0 F1F1 F1F1`	`FFFFFFF 0000000`
`FEFE FEFE FEFE FEFE`	`FFFFFFF FFFFFFF`

With all round keys equal, the key order $K_{16}, \dots, K_1$ is the same as $K_1, \dots, K_{16}$, so the reverse cipher *is* the cipher: encrypting twice with a weak key returns the plaintext, $E_K(E_K(P)) = P$.

Semi-weak keys. A **semi-weak key** creates only 2 distinct round keys, each used 8 times (each half follows the alternating pattern 0101 … or 1010 …, or is constant, with at least one half alternating). They come in **6 pairs** (k_1, k_2) whose round-key sequences are each other's reverse, so $E_{k_2}(E_{k_1}(P)) = P$: one key of the pair decrypts what the other encrypts. The pairs (all checked by script) are:

First key	Second key	First key	Second key
`01FE 01FE 01FE 01FE`	`FE01 FE01 FE01 FE01`	`1FFE 1FFE 0EFE 0EFE`	`FE1F FE1F FE0E FE0E`
`1FE0 1FE0 0EF1 0EF1`	`E01F E01F F10E F10E`	`011F 011F 010E 010E`	`1F01 1F01 0E01 0E01`
`01E0 01E0 01F1 01F1`	`E001 E001 F101 F101`	`E0FE E0FE F1FE F1FE`	`FEE0 FEE0 FEF1 FEF1`

Possible weak keys. Forouzan lists **48 possible weak keys**, which create only 4 distinct round keys, each used 4 times (halves repeating with period 4, such as 0011 0011 …). Since 2^{56} keys contain only $4 + 12 + 48 = 64$ such keys, a randomly chosen key is weak with probability 2^{-50}, so these keys matter only if they are chosen on purpose.

Key complement. Let $\bar{x}$ be the bitwise complement. Then $E_{\bar{K}}(\bar{P}) = \overline{E_K(P)}$. Proof: complementing K complements every round key, and complementing the data complements R_{i-1} and hence its expansion; then $E(\bar{R}) \oplus \bar{K}_i = E(R) \oplus K_i$, so the S-box inputs and f are unchanged, and $\bar{L} \oplus f = \overline{L \oplus f}$, so every half stays complemented through all rounds (checked by script). As a result, a chosen-plaintext attacker who has $C_1 = E_K(P)$ and $C_2 = E_K(\bar{P})$ can test two keys with one encryption (K and $\bar{K}$), halving the brute-force work to 2^{55}.

1.8 Multiple DES

The obvious fix for a short key is to encrypt several times with different keys. This only helps because DES is **not a group**: if for every K_1, K_2 there were a K_3 with $E_{K_2}(E_{K_1}(P)) = E_{K_3}(P)$, double encryption would just be single DES with another key. It has been shown that no such K_3 exists in general, so cascading DES really produces new mappings.

Double DES. $C = E_{K_2}(E_{K_1}(P))$ and $P = D_{K_1}(D_{K_2}(C))$, with a 112-bit key (K_1, K_2). One would expect 2^{112} trials for a brute-force attack, but the **meet-in-the-middle attack** (a known-plaintext attack) needs only about 2^{57}. The derivation starts from the middle text M, the value between the two encryptions:

$$M = E_{K_1}(P) \quad \text{and also} \quad M = D_{K_2}(C).$$

Given one known pair (P, C), Eve encrypts P under all 2^{56} values of K_1 and stores the results M in a table sorted by M. She then decrypts C under all 2^{56} values of K_2 and looks up each $D_{K_2}(C)$ in the table. Every match gives a candidate pair (K_1, K_2), since it makes both equations hold. The cost is $2^{56}+2^{56} = 2^{57}$ DES operations (plus memory for 2^{56} entries), instead of $2^{56} \times 2^{56} = 2^{112}$, because the two keys are searched *one after the other* rather than in all combinations. Some matches are false: 2^{112} key pairs map to only 2^{64} middle values, so about $2^{112}/2^{64} = 2^{48}$ pairs fit one (P, C) by chance; testing the candidates on a second known pair (P', C') leaves about $2^{48}/2^{64} < 1$ false survivor, so the right key is identified. Double DES raises the work only from 2^{56} to 2^{57}, not to 2^{112}.

Triple DES with two keys. $C = E_{K_1}(D_{K_2}(E_{K_1}(P)))$ and $P = D_{K_1}(E_{K_2}(D_{K_1}(C)))$, the **EDE** (encrypt–decrypt–encrypt) form. The key is 112 bits, and a meet-in-the-middle attack now has to split three stages into one and two, which leaves a 2^{112} search on one side, so the attack no longer helps. The middle stage is a *decryption* for compatibility: with $K_1 = K_2$ the first two stages cancel and 3DES becomes single DES, so a 3DES device can talk to an old DES device (checked by script). Two-key 3DES is used in PEM and in the financial standard ANS X9.17 / ISO 8732.

Triple DES with three keys. $C = E_{K_3}(D_{K_2}(E_{K_1}(P)))$ with a 168-bit key. Meet-in-the-middle reduces its effective strength to about 2^{112}, which is still far beyond brute force; it is used in PGP and S/MIME.

1.9 Security of DES in one line each

- **Brute force:** 2^{56} keys (2^{55} with the complement property), feasible with special hardware or networked computers.
- **Differential cryptanalysis:** DES can be broken with 2^{47} chosen plaintexts; its S-boxes were evidently designed against this attack, which is why it is no better than brute force in practice.
- **Linear cryptanalysis:** DES can be broken with 2^{43} known plaintexts, a newer attack the designers did not anticipate.

Corrections to the notes / slides

- The slides' **substitute** pseudocode (L8 slide 16, as in Forouzan) declares inBlock[32], outBlock[48]; the sizes are swapped (48 bits in, 32 bits out). It also indexes inBlock[$i \times 6 + 1$] with $i = 1..8$, which reaches position 49 for $i = 8$; the group offset must be $(i - 1) \times 6$ (and $(i - 1) \times 4$ for outBlock), as in the corrected version in the Q6 answer (verified by script).
- Semi-weak key table (L8 slide 23), row 3: `01E0 01E1 01F1 01F1` should be `01E0 01E0 01F1 01F1`. `E1` differs from `E0` only in the parity bit, so it is the same 56-bit key, but it violates odd parity.
- Key-size slide (L8 slide 21): "In 1977 a team of researchers used 3500 computers … RSA Laboratories" should read 1997 (RSA's DES Challenge); DES itself was only published in 1977.

Check your understanding

1. **Why do IP and FP add no security?** They are fixed, keyless and public, so an attacker can apply them himself and attack the 16 rounds directly.
2. **Why does round 16 have no swapper?** So that the sequence of operations of the reverse cipher is identical to the cipher; then decryption is the same algorithm with keys $K_{16}, \ldots, K_1$.
3. **What is the output of S-box 1 for input `111111`?** Row $(11)_2 = 3$, column $(1111)_2 = 15$; S1[3][15] = 13 = `1101`.
4. **Why does DES need an expansion P-box?** To match the 48-bit round key and to give eight 6-bit S-box inputs; the duplicated edge bits also feed two neighbouring S-boxes, which speeds up the avalanche.
5. **How many bits are shifted in total, and why does it matter?** $4 \times 1 + 12 \times 2 = 28$, a full rotation, so $C_{16} = C_0$ and $D_{16} = D_0$ and decryption keys can be generated by right shifts from the original halves.
6. **Is the DES S-box invertible? Does it matter?** No, a $6 \to 4$ box has 4 preimages per output; it does not matter because it sits inside f, which a Feistel cipher never inverts.
7. **What is special about a weak key?** All 16 round keys are equal, so $E_K(E_K(P)) = P$; there are 4 weak keys, 6 pairs of semi-weak keys and 48 possible weak keys.
8. **Why is double DES not 112-bit secure?** Meet-in-the-middle: with one known (P, C), tabulate $E_{K_1}(P)$ for all 2^{56} keys and match against $D_{K_2}(C)$ for all 2^{56} keys, which costs 2^{57} operations instead of 2^{112}.
9. **Why is triple DES EDE and not EEE?** With $K_1 = K_2$ (or $K_2 = K_3$), EDE collapses to single DES, so new equipment stays compatible with single-DES equipment.
10. **State the complement property and its consequence.** $E_{\bar{K}}(\bar{P}) = \overline{E_K(P)}$, so a chosen-plaintext brute-force attack

tests two keys per encryption and needs only 2^{55} trials.